Name : Samiksha Gaiki
Post Test Theory Questions

Answers:

Q1
Constructor called
Printed first because Spring creates the MyBean object when the application starts, so the constructor executes first.

Init called
Printed after the constructor because methods annotated with @PostConstruct are executed by Spring after bean creation and dependency injection are completed.

Q2

Mistake:
The ResponseEntity object is created using only HttpStatus.OK, but the actual User object (u) is not passed in the response body.
Because of this, the API returns status 200 OK with an empty body.
Corrected Line:
return new ResponseEntity<>(u, HttpStatus.OK);

Q4

The transfer() method should be annotated with @Transactional.

@Transactional
public void transfer(Long fromId, Long toId, double amount)

The reason for using @Transactional is that money transfer is a single logical operation involving multiple database updates. In this method, money is deducted from one account and added to another account. Both operations must either complete successfully together or fail together.
If @Transactional is missing, a problem can occur when one database operation succeeds and the other fails. For example, the amount may get deducted from the sender's account, but due to an

exception before saving the receiver's account, the credited amount may not be updated. This leads to inconsistent and incorrect bank data.
With @Transactional, Spring treats the entire method as one transaction. If any exception occurs in between, all changes are rolled back automatically, keeping the database consistent and safe.

Q6

This JWT filter checks whether the incoming request contains a valid JWT token in the Authorization header. If the token is valid, it extracts the username from the token and sets the authentication in Spring Security's SecurityContextHolder, allowing the user to access secured APIs.

The bug is in this line:
String token = header.substring(6);
The Bearer prefix contains 7 characters (including the space), but the code removes only 6 characters. Because of this, the extracted token still contains a leading space and becomes invalid.
Corrected Line:
String token = header.substring(7);

Q7

@Query("SELECT p FROM Product p WHERE p.price < :price AND p.category = :category")
List<Product> findProducts(@Param("price") double price, @Param("category") String category);

Q8

The N+1 query problem happens when JPA executes one query to fetch the main data and then executes additional queries for related data one by one. This increases the number of database calls and reduces performance.
For example, suppose we have a Department entity and each department has many employees. First, JPA fetches all departments using one query. Then, for every department, it separately fetches employees. So if there are 5 departments, JPA runs 1 query for departments + 5 extra queries for employees. This becomes the N+1 problem.

The most common cause is using lazy loading (FetchType.LAZY) on relationships and then accessing related entities inside a loop.
Fixed Line:
@Query("SELECT d FROM Department d JOIN FETCH d.employees")

Q9

SQL Query

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

JPQL

```
@Query("SELECT MAX(e.salary) FROM Employee e WHERE e.salary < (SELECT
MAX(e2.salary) FROM Employee e2)")
double findSecondHighestSalary();
```

Q10

```
List<Student> findByCity(String city);
List<Student> findByNameAndCity(String name, String city);
List<Student> findByEmailContaining(String keyword);
long countByCity(String city);
```

Q11

This problem is called the N+1 Query Problem.
Here, findAll() executes 1 query to fetch all orders. Then, inside the loop, o.getItems() triggers separate queries for each order to fetch items. So for 10 orders, total queries become 11.

Line causing the problem:
o.getItems().size()
This line loads items lazily for every order.

One-line Fix using JOIN FETCH:
@Query("SELECT o FROM Order o JOIN FETCH o.items")
This fetches orders and their items together in a single query, avoiding extra database calls.

Q16

a) Benefits of splitting into microservices

1. Each service can be developed and deployed independently, which makes development faster.

2. Different services can scale separately based on demand. For example, the Order service can be scaled without scaling the User service.
3. If one service fails, the entire application does not stop working, improving fault isolation and reliability.

(b) Challenges of microservices

1. Managing communication between multiple services becomes more complex.
2. Deployment, monitoring, and debugging are harder compared to a monolithic application.

(c) How Eureka and API Gateway work together
Eureka acts as a Service Discovery server where all microservices like User Service, Order Service, and Product Service register themselves. The API Gateway also connects to Eureka and knows the locations of all services dynamically.
The React frontend sends all requests to a single API Gateway URL. The API Gateway then forwards each request to the correct microservice using service names from Eureka. This removes the need for the frontend to know the IP addresses or ports of individual services.

Q21

```
async function loadUsers() {

  try {

    const response = await fetch("/api/users");

    if (!response.ok) {
      throw new Error("Failed to fetch users");
    }

    const users = await response.json();

    const userList = document.getElementById("user-list");

    users.forEach(user => {

      const li = document.createElement("li");
      li.textContent = user.name;

      userList.appendChild(li);
    });
```

```
    } catch (error) {

        document.getElementById("error").textContent =
            "Error loading users";
    }
}
```

Q22

a)The useEffect() does not have a dependency array.
Because of this, the effect runs after every render. Inside the effect, setUsers() updates the state, which causes another render, creating an infinite loop.

b)+c)

```
import { useEffect, useState } from "react";

function UserList() {

    const [users, setUsers] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {

        fetch('/api/users')
            .then(res => res.json())
            .then(data => {
                setUsers(data);
                setLoading(false);
            });

    }, []);

    if (loading) {
        return <p>Loading...</p>;
    }

    return (
        <ul>
            {users.map(u => (
                <li key={u.id}>{u.name}</li>
```

```
            ))}
        </ul>
    );
}
```

Note:Remaining theory answers are there in “TheoryAnswer” named file in the respective package